This is a copy of a shared ChatGPT chat. It won't be added to memory.
**Report conversation**

**Today** 8:25 AM

# Arm Wrestle Showdown Lab

This project is a tug-of-war button mashing game using **Pygame**. It introduces students to vector interpolation, resource/stamina management, alternating key-stroke detection, and simple AI pressure modeling inside an object-oriented codebase.

---

## What's Provided

A working Arm Wrestling game with:

- An arm wrestling table arena rendering procedural arms, elbows, shoulders, and clasping hands
- An alternating input mechanism requiring rhythmic pressing of Left and Right arrow keys
- A stamina bar that depletes during rapid pressing and recovers naturally over time
- Continuous computer AI force pushing toward the player's side
- Win/loss boundary detection and a post-game rematch screen

It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

## Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for

**this lab.**

---

## Getting Started

### Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
pip install pygame
```

3. Run the game:

```
python main.py
```

**Controls:** Rapidly alternate Left Arrow and Right Arrow to push, R to restart after a match.

## Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

### Task 1: Fix the inverted arm push bug

To win the match, the player needs to pull arm_position toward negative values (<= -self.target_limit). In the current build, when the player alternates Left and Right arrow keys, the input handler adds +4.2 to self.arm_position instead of subtracting it. This helps the computer pin the player instead of resisting. Correct the sign operation so player inputs push the arm toward the player's winning threshold.

### Task 2: Implement dynamic AI surge / difficulty spikes

Currently, the AI applies force at a constant average rate using static math (self.ai_strength * ai_variance).

Implement an AI stamina or surge mechanism in game_engine.update() where the computer builds up energy, periodically triggers a "power surge" with increased force for 1–2 seconds, and then enters an exhausted state with reduced resistance, creating a dynamic back-and-forth rhythm.

### Task 3: Implement an exhaustion warning indicator

When the player drops below 10 stamina, button inputs are disabled until stamina regenerates. However, there is no immediate visual cue explaining why inputs stopped working. Add visual feedback—such as flashing the stamina bar red, displaying an "EXHAUSTED!" label, or causing the player's arm to tremble—whenever stamina is below the usable threshold of 10.

### Task 4: Implement an exhaustion warning indicator

When the player drops below 10 stamina, button inputs are disabled until stamina regenerates. However, there is no immediate visual cue explaining why inputs stopped working. Add visual feedback—such as flashing the stamina bar red, displaying an "EXHAUSTED!" label, or causing the player's arm to tremble—whenever stamina is below the usable threshold of 10.

---

## Expected Behavior

- Rapidly alternating between Left Arrow and Right Arrow pulls the hands toward the player's side.
- Rapid pushing drains stamina; falling below 10 stamina temporarily halts input until it recovers.

- Reaching -100.0 awards victory to the player, while reaching +100.0 triggers a computer win.
- Pressing R on the Game Over screen resets stamina, arm position, and game states for a rematch.

---

## Folder Structure

```
arm_wrestle/
├── game/
│   └── game_engine.py
├── main.py
└── README.md
```

---

## Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

Instructions:

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code

7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.
10. Do not raise PR to the main repo (SETAPESU26)

help me

Absolutely. This lab is quite manageable. The important part is that your **submission is not asking for a report**—you mainly need:

1. **10-sec video before changes**
2. **Fix Task 1 + implement Tasks 2–4**
3. **10-sec video after changes**
4. **Complete ChatGPT chat-history link**
5. **Push your modified code to your assigned repo**
6. **Do NOT create a PR to** `SETAPESU26`

We can do this step-by-step, including the **LLM interaction itself**, so your chat history clearly shows how you used ChatGPT as your debugging/pair-programming partner.

## One important observation

Tasks **3 and 4 are identical** in the README you pasted:

> Task 3: Implement an exhaustion warning indicator
> Task 4: Implement an exhaustion warning indicator

So you don't need to invent a separate fourth feature unless the actual repository README contains something different.

## Let's do it safely

First, **don't modify the code yet**.

Send me either:

- the contents of `game/game_engine.py` and `main.py`, **or**

- upload the repository/project folder/files here.

Then I'll help you in this order:

**Step 1 → Run the original game**
We'll verify the existing behavior and decide exactly what to show in your **before video**.

**Step 2 → Task 1**
I'll identify the exact line causing the inverted arm movement and explain the sign change.

**Step 3 → Task 2**
We'll add a proper AI surge system:

- AI builds energy
- surge starts periodically
- surge lasts ~1–2 seconds
- AI becomes stronger during surge
- AI becomes temporarily exhausted afterward
- normal resistance returns

**Step 4 → Task 3/4**
We'll add a clear `"EXHAUSTED!"` visual warning and make sure it activates when stamina `< 10`.

**Step 5 → Test everything**
We'll specifically test:

- Left/Right alternating → arm moves toward `-100`
- stamina decreases
- stamina `< 10` → inputs stop
- exhaustion warning appears
- stamina recovers
- AI surge occurs
- AI exhaustion occurs
- `-100` → player wins
- `+100` → computer wins

- `R` → complete rematch reset

**Step 6 → After video**
I'll tell you exactly what to demonstrate in the 10-second recording.

**Step 7 → Git commands**
I'll give you the exact commands to commit and push to **your assigned repo**, without touching the main `SETAPESU26` repository.

Send me `game/game_engine.py` **first** (and `main.py` if it is relevant). We should work with your actual code rather than guessing the structure.

game/game_engine.py

```
import math
import random
import pygame


class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)
```

```
    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and
event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.stamina <= 10:
                return

            # BUG SYMPTOM:
            # Adding to arm_position pushes it toward the
COMPUTER instead of reducing it to win.
            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position += 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT
            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position += 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_RIGHT

    def update(self):
        if self.game_state != "PLAYING":
            return

        ai_variance = random.uniform(0.3, 1.0)
        self.arm_position += self.ai_strength * ai_variance

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina,
self.stamina + 0.8)

        if self.arm_position <= -self.target_limit:
```

```
                self.winner = "PLAYER"
                self.game_state = "GAME_OVER"
            elif self.arm_position >= self.target_limit:
                self.winner = "COMPUTER"
                self.game_state = "GAME_OVER"

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render("ARM WRESTLE
SHOWDOWN", True, (240, 240, 240))
        screen.blit(title_surf, (self.width // 2 -
title_surf.get_width() // 2, 12))

        player_header = self.font_med.render("PLAYER",
True, (80, 160, 255))
        computer_header =
self.font_med.render("COMPUTER", True, (255, 100, 80))
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))

        table_rect = pygame.Rect(40, 100, self.width - 80,
310)
        pygame.draw.rect(screen, (110, 50, 15), table_rect,
border_radius=14)
        pygame.draw.rect(screen, (70, 30, 8), table_rect,
width=5, border_radius=14)

        pygame.draw.line(screen, (45, 18, 4), (self.width //
2, 100), (self.width // 2, 410), 4)
```

```
        offset_x = (self.arm_position / self.target_limit) * 95
        hand_x = (self.width // 2) + int(offset_x)
        hand_y = 235

        p_shoulder = (70, 330)
        p_elbow = (140, 215)
        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 215)

        pygame.draw.line(screen, (200, 145, 110),
p_shoulder, p_elbow, 32)
        pygame.draw.line(screen, (215, 160, 125), p_elbow,
(hand_x, hand_y), 26)
        pygame.draw.circle(screen, (185, 130, 95),
p_elbow, 18)

        pygame.draw.line(screen, (170, 110, 85),
c_shoulder, c_elbow, 32)
        pygame.draw.line(screen, (185, 125, 95), c_elbow,
(hand_x, hand_y), 26)
        pygame.draw.circle(screen, (150, 95, 70), c_elbow,
18)

        pygame.draw.circle(screen, (225, 175, 140),
(hand_x, hand_y), 24)
        pygame.draw.circle(screen, (160, 115, 85), (hand_x,
hand_y), 24, width=3)

        stamina_label = self.font_med.render("STAMINA",
True, (220, 220, 220))
        screen.blit(stamina_label, (40, 445))

        stamina_bg = pygame.Rect(140, 448, 240, 22)
        stamina_fill = pygame.Rect(140, 448, int(240 *
(self.stamina / self.max_stamina)), 22)
        pygame.draw.rect(screen, (45, 50, 60), stamina_bg,
border_radius=6)
        bar_color = (60, 210, 100) if self.stamina > 25 else
```

```
(220, 60, 60)
        pygame.draw.rect(screen, bar_color, stamina_fill,
border_radius=6)

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface((self.width,
self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            win_text = "PLAYER WINS THE MATCH!" if
self.winner == "PLAYER" else "COMPUTER WINS!"
            color = (80, 240, 100) if self.winner == "PLAYER"
else (240, 80, 80)
            text_surf = self.font_big.render(win_text, True,
color)
            screen.blit(
                text_surf,
                (self.width // 2 - text_surf.get_width() // 2,
self.height // 2 - 45)
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch", True, (240, 240, 240)
            )
            screen.blit(
                restart_surf,
                (self.width // 2 - restart_surf.get_width() // 2,
self.height // 2 + 10)
            )

main.py
import sys
import pygame
from game.game_engine import GameEngine

SCREEN_WIDTH = 600
SCREEN_HEIGHT = 620
```

```
FPS = 60


def main():
    pygame.init()
    pygame.display.set_caption("Arm Wrestle
Showdown")
    screen = pygame.display.set_mode((SCREEN_WIDTH,
SCREEN_HEIGHT))
    clock = pygame.time.Clock()

    engine = GameEngine(SCREEN_WIDTH,
SCREEN_HEIGHT)

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            else:
                engine.handle_event(event)

        engine.update()
        engine.render(screen)
        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()
    sys.exit()


if __name__ == "__main__":
    main()
```

Worked for 8s >

There are **3 actual code changes** needed here:

- **Task 1:** change both `+= 4.2` to `-= 4.2`.
- **Task 2:** add AI energy/surge/exhaustion state to `__init__()` and `update()`.
- **Task 3/4:** add an exhaustion warning in `render()` when stamina `< 10`.

One small improvement: I'll also make the AI surge visible on-screen, because that makes your **after video much easier to demonstrate**.

## 1. Replace `game/game_engine.py` with this

Completed game_engine.py

```
import math
import random
import pygame


class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        # AI surge system
        self.ai_energy = 0.0
        self.ai_max_energy = 100.0
        self.ai_surge_active = False
        self.ai_surge_timer = 0.0
        self.ai_exhausted = False
```

```
        self.ai_exhausted_timer = 0.0

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.stamina <= 10:
                return

            # Player pushes the arm toward the negative/player side.
            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT

            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = event.key

    def update(self):
        if self.game_state != "PLAYING":
            return

        # ----------------------------------------
        # AI SURGE / EXHAUSTION SYSTEM
        # ----------------------------------------

        # AI builds energy gradually.
        if not self.ai_surge_active and not self.ai_exhausted:
```

```
            self.ai_energy = min(self.ai_max_energy, self.ai_energy + 0.45

            # Once enough energy is built, trigger a power surge.
            if self.ai_energy >= self.ai_max_energy:
                self.ai_surge_active = True

                # Surge lasts between 1 and 2 seconds.
                self.ai_surge_timer = random.uniform(60, 120)

        # Power surge: stronger AI pressure.
        if self.ai_surge_active:
            ai_force = self.ai_strength * random.uniform(2.0, 3.0)
            self.arm_position += ai_force

            self.ai_surge_timer -= 1

            if self.ai_surge_timer <= 0:
                self.ai_surge_active = False
                self.ai_exhausted = True

                # Exhaustion lasts roughly 1.5 seconds.
                self.ai_exhausted_timer = 90
                self.ai_energy = 0.0

        # AI exhaustion: reduced resistance.
        elif self.ai_exhausted:
            ai_force = self.ai_strength * random.uniform(0.05, 0.15)
            self.arm_position += ai_force

            self.ai_exhausted_timer -= 1

            if self.ai_exhausted_timer <= 0:
                self.ai_exhausted = False

        # Normal AI behavior.
        else:
            ai_variance = random.uniform(0.3, 1.0)
            self.arm_position += self.ai_strength * ai_variance
```

```
        # ----------------------------------------
        # PLAYER STAMINA RECOVERY
        # ----------------------------------------

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + 0.8)

        # ----------------------------------------
        # WIN / LOSS DETECTION
        # ----------------------------------------

        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"

        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None

        self.winner = None
        self.game_state = "PLAYING"

        # Reset AI surge system
        self.ai_energy = 0.0
        self.ai_surge_active = False
        self.ai_surge_timer = 0.0
        self.ai_exhausted = False
        self.ai_exhausted_timer = 0.0

    def render(self, screen):
        screen.fill((25, 28, 35))
```

```
        title_surf = self.font_big.render(
            "ARM WRESTLE SHOWDOWN",
            True,
            (240, 240, 240)
        )
        screen.blit(
            title_surf,
            (self.width // 2 - title_surf.get_width() // 2, 12)
        )

        player_header = self.font_med.render(
            "PLAYER",
            True,
            (80, 160, 255)
        )

        computer_header = self.font_med.render(
            "COMPUTER",
            True,
            (255, 100, 80)
        )

        screen.blit(player_header, (60, 55))
        screen.blit(
            computer_header,
            (self.width - 150, 55)
        )

        # ----------------------------------------
        # AI STATUS INDICATOR
        # ----------------------------------------

        if self.ai_surge_active:
            ai_status = self.font_med.render(
                "AI POWER SURGE!",
                True,
                (255, 180, 50)
            )
```

```
            screen.blit(
                ai_status,
                (
                    self.width // 2 - ai_status.get_width() // 2,
                    55
                )
            )

        elif self.ai_exhausted:
            ai_status = self.font_med.render(
                "AI EXHAUSTED",
                True,
                (100, 220, 255)
            )

            screen.blit(
                ai_status,
                (
                    self.width // 2 - ai_status.get_width() // 2,
                    55
                )
            )

        # ----------------------------------------
        # TABLE
        # ----------------------------------------

        table_rect = pygame.Rect(
            40,
            100,
            self.width - 80,
            310
        )

        pygame.draw.rect(
            screen,
            (110, 50, 15),
```

```
            table_rect,
            border_radius=14
        )

        pygame.draw.rect(
            screen,
            (70, 30, 8),
            table_rect,
            width=5,
            border_radius=14
        )

        pygame.draw.line(
            screen,
            (45, 18, 4),
            (self.width // 2, 100),
            (self.width // 2, 410),
            4
        )

        offset_x = (
            self.arm_position / self.target_limit
        ) * 95

        hand_x = (
            self.width // 2
        ) + int(offset_x)

        hand_y = 235

        p_shoulder = (70, 330)
        p_elbow = (140, 215)

        c_shoulder = (
            self.width - 70,
            330
        )
```

```
        c_elbow = (
            self.width - 140,
            215
        )

        pygame.draw.line(
            screen,
            (200, 145, 110),
            p_shoulder,
            p_elbow,
            32
        )

        pygame.draw.line(
            screen,
            (215, 160, 125),
            p_elbow,
            (hand_x, hand_y),
            26
        )

        pygame.draw.circle(
            screen,
            (185, 130, 95),
            p_elbow,
            18
        )

        pygame.draw.line(
            screen,
            (170, 110, 85),
            c_shoulder,
            c_elbow,
            32
        )

        pygame.draw.line(
            screen,
```

```
            (185, 125, 95),
            c_elbow,
            (hand_x, hand_y),
            26
        )

        pygame.draw.circle(
            screen,
            (150, 95, 70),
            c_elbow,
            18
        )

        pygame.draw.circle(
            screen,
            (225, 175, 140),
            (hand_x, hand_y),
            24
        )

        pygame.draw.circle(
            screen,
            (160, 115, 85),
            (hand_x, hand_y),
            24,
            width=3
        )

        # ----------------------------------------
        # STAMINA BAR
        # ----------------------------------------

        stamina_label = self.font_med.render(
            "STAMINA",
            True,
            (220, 220, 220)
        )
```

```
        screen.blit(
            stamina_label,
            (40, 445)
        )

        stamina_bg = pygame.Rect(
            140,
            448,
            240,
            22
        )

        stamina_fill = pygame.Rect(
            140,
            448,
            int(
                240 *
                (self.stamina / self.max_stamina)
            ),
            22
        )

        pygame.draw.rect(
            screen,
            (45, 50, 60),
            stamina_bg,
            border_radius=6
        )

        if self.stamina <= 10:
            # Flashing exhaustion warning.
            if (pygame.time.get_ticks() // 250) % 2 == 0:
                bar_color = (255, 30, 30)
            else:
                bar_color = (120, 30, 30)
        elif self.stamina <= 25:
            bar_color = (220, 60, 60)
        else:
```

```
            bar_color = (60, 210, 100)

        pygame.draw.rect(
            screen,
            bar_color,
            stamina_fill,
            border_radius=6
        )

        # ----------------------------------------
        # EXHAUSTION WARNING
        # ----------------------------------------

        if self.stamina <= 10:
            warning_text = self.font_big.render(
                "EXHAUSTED!",
                True,
                (255, 60, 60)
            )

            screen.blit(
                warning_text,
                (
                    self.width // 2 -
                    warning_text.get_width() // 2,
                    490
                )
            )

            help_text = self.font_med.render(
                "Wait for stamina to recover...",
                True,
                (240, 200, 200)
            )

            screen.blit(
                help_text,
                (
```

```
                self.width // 2 -
                help_text.get_width() // 2,
                535
            )
        )

    # ----------------------------------------
    # GAME OVER SCREEN
    # ----------------------------------------

    if self.game_state == "GAME_OVER":
        overlay = pygame.Surface(
            (self.width, self.height),
            pygame.SRCALPHA
        )

        overlay.fill((0, 0, 0, 200))
        screen.blit(
            overlay,
            (0, 0)
        )

        win_text = (
            "PLAYER WINS THE MATCH!"
            if self.winner == "PLAYER"
            else "COMPUTER WINS!"
        )

        color = (
            (80, 240, 100)
            if self.winner == "PLAYER"
            else (240, 80, 80)
        )

        text_surf = self.font_big.render(
            win_text,
            True,
            color
```

```
            )

            screen.blit(
                text_surf,
                (
                    self.width // 2 -
                    text_surf.get_width() // 2,
                    self.height // 2 - 45
                )
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch",
                True,
                (240, 240, 240)
            )

            screen.blit(
                restart_surf,
                (
                    self.width // 2 -
                    restart_surf.get_width() // 2,
                    self.height // 2 + 10
                )
            )
```

## 2. What exactly changed?

### Task 1 — Bug fix

Original:

```
self.arm_position += 4.2
```

Now:

```
self.arm_position -= 4.2
```

This is the key fix.

Your coordinate system is:

```
PLAYER                          COMPUTER
  -100          0                  +100
   ←-------------|----------------→
                 |
             starting
              point
```

Therefore:

- Player press → `arm_position` decreases
- Computer force → `arm_position` increases
- Player wins at `-100`
- Computer wins at `+100`

---

## Task 2 — AI surge

The AI now has:

```
self.ai_energy
self.ai_surge_active
self.ai_surge_timer
self.ai_exhausted
self.ai_exhausted_timer
```

The cycle is:

```
Normal AI
   ↓
Build AI energy
   ↓
Energy reaches 100
   ↓
POWER SURGE
   ↓
Strong AI force for 1–2 sec
   ↓
```

```
AI EXHAUSTED
   ↓
Very weak AI force
   ↓
Normal AI
   ↓
Repeat
```

This gives the game the requested **back-and-forth rhythm** instead of a constant AI force.

---

### Task 3/4 — Exhaustion warning

When:

```
self.stamina <= 10
```

the player gets:

```
       EXHAUSTED!
Wait for stamina to recover...
```

and the stamina bar flashes.

Your existing code already prevents input:

```
if self.stamina <= 10:
    return
```

So we don't need to change that behavior.

---

## 3. Before making these changes — DO THIS FIRST

This is important for your submission.

Run your **original code** before replacing it.

From the project root:

```
python main.py
```

If `pygame` isn't installed:

```
pip install pygame
```

Then record approximately **10 seconds**.

### What should your BEFORE video show?

Start the game and press:

```
LEFT → RIGHT → LEFT → RIGHT → LEFT → RIGHT
```

rapidly.

Because of the bug, you'll see the arm moving toward the **computer side**, even though you're trying to push as the player.

That is exactly what you want to capture.

Don't worry if you don't manage to demonstrate every existing behavior in 10 seconds. The purpose of this video is simply to establish the **before state**.

---

## 4. After replacing the code

Run:

```
python main.py
```

Test these things manually.

### Test 1 — Player movement

Press:

```
LEFT
RIGHT
LEFT
RIGHT
```

The hand should now move:

```
PLAYER
  ←
  |
  ●
  |
  →
COMPUTER
```

More precisely, the value should become increasingly negative.

---

## Test 2 — Exhaustion

Rapidly alternate:

`LEFT RIGHT LEFT RIGHT ...`

Each valid press consumes:

`2.0 stamina`

When stamina reaches `10`:

```
EXHAUSTED!
Wait for stamina to recover...
```

should appear.

The stamina bar should flash.

Try pressing the arrows while exhausted—the arm should **not respond to your inputs**.

After waiting, stamina should recover automatically.

---

## Test 3 — AI surge

You should eventually see:

```
AI POWER SURGE!
```

The computer should push noticeably harder.

After the surge:

```
AI EXHAUSTED
```

should appear, and the AI should become much weaker temporarily.

This is particularly useful for your after-video because the evaluator can visually see that your feature exists.

---

### Test 4 — Rematch

Let either side win.

You should see:

```
PLAYER WINS THE MATCH!
```

or:

```
COMPUTER WINS!
```

Then press:

```
R
```

Everything should return to the starting state.

---

## 5. One thing I want you to check before submitting

There's a tiny issue in the README wording:

> "three optional features"

but the listed tasks are:

1. Bug fix
2. AI surge
3. Exhaustion indicator
4. Exhaustion indicator again

So **don't invent another feature yet**. Your actual repository may have a different README version, but based on the README you pasted, Task 3 and Task 4 are duplicates.

If your instructor expects exactly the repository README, follow **the README inside your cloned repository** as the authoritative version.

---

## 6. Your LLM chat history

The assignment explicitly says:

> "Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner"

and asks for:

> "The Chat/LLM used page link, with the complete chat history"

**Keep this conversation.** Don't start a separate ChatGPT conversation for the implementation unless necessary.

Your current conversation already demonstrates:

- you provided the source code
- you identified the bug
- we analyzed the code
- we implemented the requested features
- we can test/debug the implementation iteratively

So this conversation can serve as your LLM interaction history.

---

## One important next step

**Don't push this code yet.**

First run it and tell me exactly what happens—especially if you get an error. If it runs, we can do a **feature-by-feature test together** and I'll help you make sure the after-video clearly demonstrates all the required changes.

---

Here's a quick summary of the conversation:

This conversation discusses an arm wrestling game project built with Pygame, focusing on debugging and feature implementation. The main issues include fixing a bug where player input incorrectly moves the arm position, adding a dynamic AI surge mechanic to create variable difficulty, and implementing a visual exhaustion warning when player stamina is low. The assistant provides detailed code fixes and walks through planned steps to test and demonstrate changes for submission.

- ↳ Ask how to test the AI surge and exhaustion features during gameplay
- ↳ Request guidance on recording the before and after gameplay videos
- ↳ Inquire about potential improvements or additional features beyond the lab tasks

ChatGPT can make mistakes. Check important info.